

TP 3 PROGRAMACIÓN AVANZADA

ALUMNO: Romero Gonzalo

Repositorio github usado:

<https://github.com/gonzafc/tps-programaci-n-avanzada/tree/main/TP3>

10.1 Preparación del entorno

1. Cloná o copiá el proyecto base entregado (carpeta 02-todo-api-postgres/).

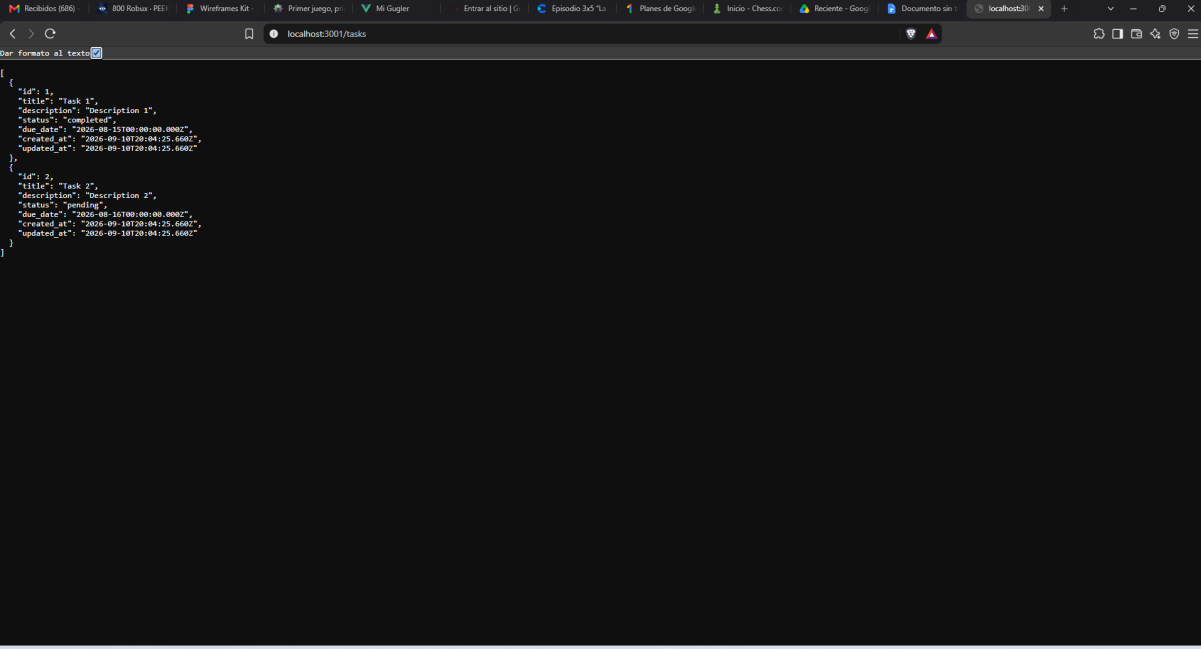
<https://github.com/erneledesma/02-todo-api-postgres>

2. Copiá las variables de entorno: `cp .env.example .env`

3. Levantá los servicios: `docker compose up --build`

4. Verificá que la API responde en `http://localhost:3001/tasks` y que devuelve las dos tareas de ejemplo cargadas por `init.sql`.

La api responde en `/tasks`:



```
{
  "id": 1,
  "title": "Task 1",
  "description": "Description 1",
  "status": "completed",
  "due_date": "2026-08-15T00:00:00.000Z",
  "created_at": "2026-09-10T20:04:25.660Z",
  "updated_at": "2026-09-10T20:04:25.660Z"
},
{
  "id": 2,
  "title": "Task 2",
  "description": "Description 2",
  "status": "pending",
  "due_date": "2026-08-16T00:00:00.000Z",
  "created_at": "2026-09-10T20:04:25.660Z",
  "updated_at": "2026-09-10T20:04:25.660Z"
}
```

10.2 Ejercicio 1 — Implementar PUT `/tasks/:id` (RF-04)

Completá el handler en `server.js`. La actualización debe conservar los valores existentes en los campos que no vengan en el body, usando COALESCE en la sentencia UPDATE, y debe actualizar `updated_at` a la fecha/hora actual. Si el `id` no existe, la respuesta debe ser 404.

Se modificó el código:

```
app.put('/tasks/:id', async (req, res) => {
  try {
    const { id } = req.params
    const { title, description, status, dueDate } = req.body

    const result = await pool.query(
      `UPDATE tasks
      SET title = COALESCE($1, title),
        description = COALESCE($2, description),
        status = COALESCE($3, status),
        due_date = COALESCE($4, due_date),
        updated_at = NOW()
      WHERE id = $5
      RETURNING *`,
      [title || null, description || null, status || null, dueDate || null, id]
    )

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Task not found' })
    }

    res.json(result.rows[0])
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
})
```

antes de modificar la task con id 1:

Dar formato al texto ☒

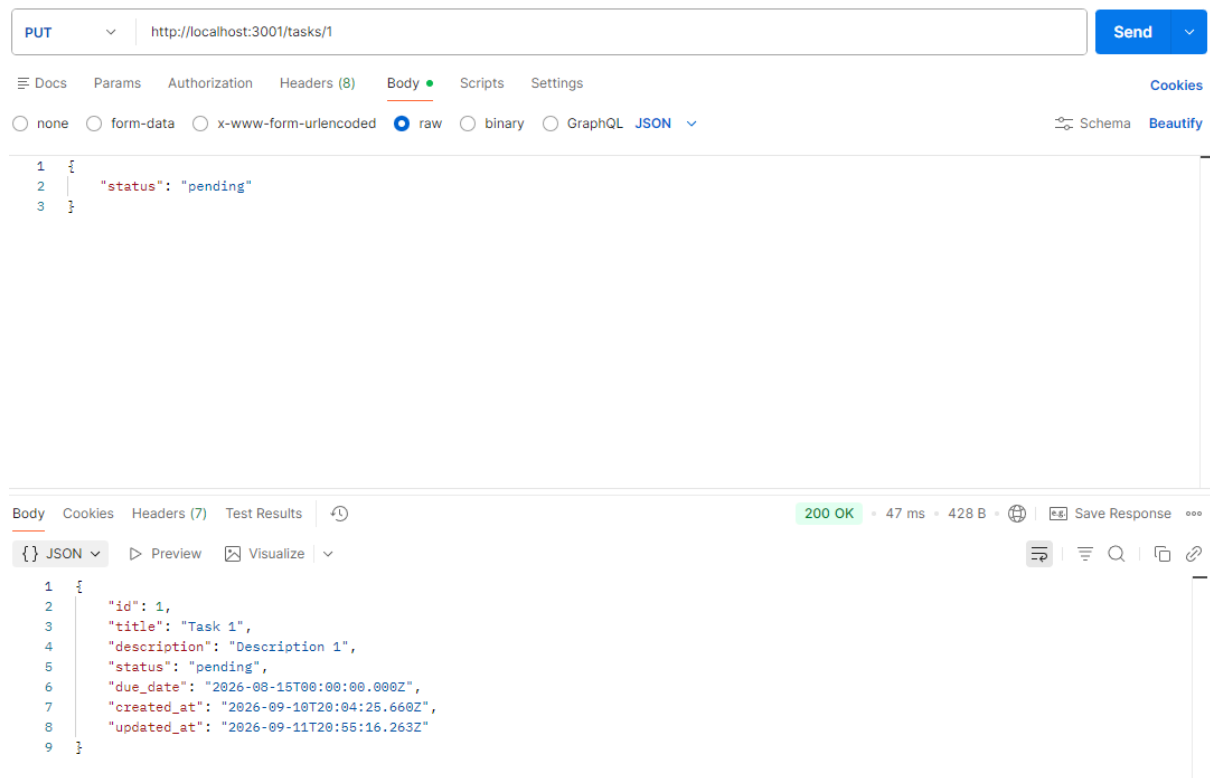
```
[
  {
    "id": 1,
    "title": "Task 1",
    "description": "Description 1",
    "status": "completed",
    "due_date": "2026-08-15T00:00:00.000Z",
    "created_at": "2026-09-10T20:04:25.660Z",
    "updated_at": "2026-09-10T20:04:25.660Z"
  },
]
```

después de modificar la task con id 1:

Dar formato al texto ☒

```
[
  {
    "id": 1,
    "title": "Task 1",
    "description": "Description 1",
    "status": "pending",
    "due_date": "2026-08-15T00:00:00.000Z",
    "created_at": "2026-09-10T20:04:25.660Z",
    "updated_at": "2026-09-11T20:55:16.263Z"
  },
]
```

en postman:



10.3 Ejercicio 2 — Implementar DELETE /tasks/:id (RF-05)

Completá el handler de eliminación usando DELETE ... RETURNING * para poder distinguir si la fila existía. Respuesta 204 sin cuerpo si se eliminó, 404 si no existía. código:

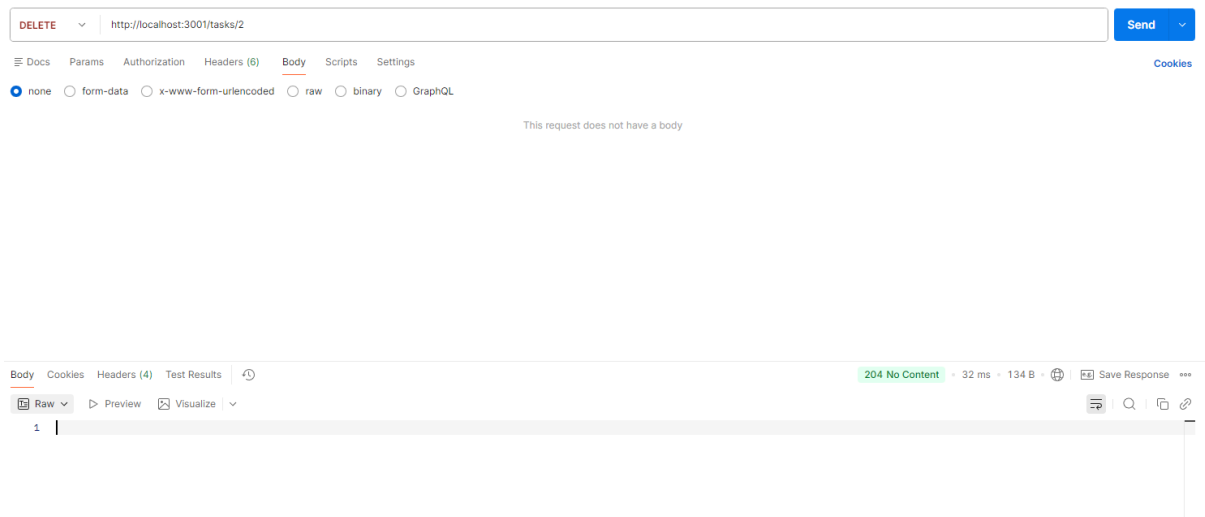
```
app.delete('/tasks/:id', async (req, res) => {
  try {
    const { id } = req.params

    const result = await pool.query(
      'DELETE FROM tasks WHERE id = $1 RETURNING *',
      [id]
    )

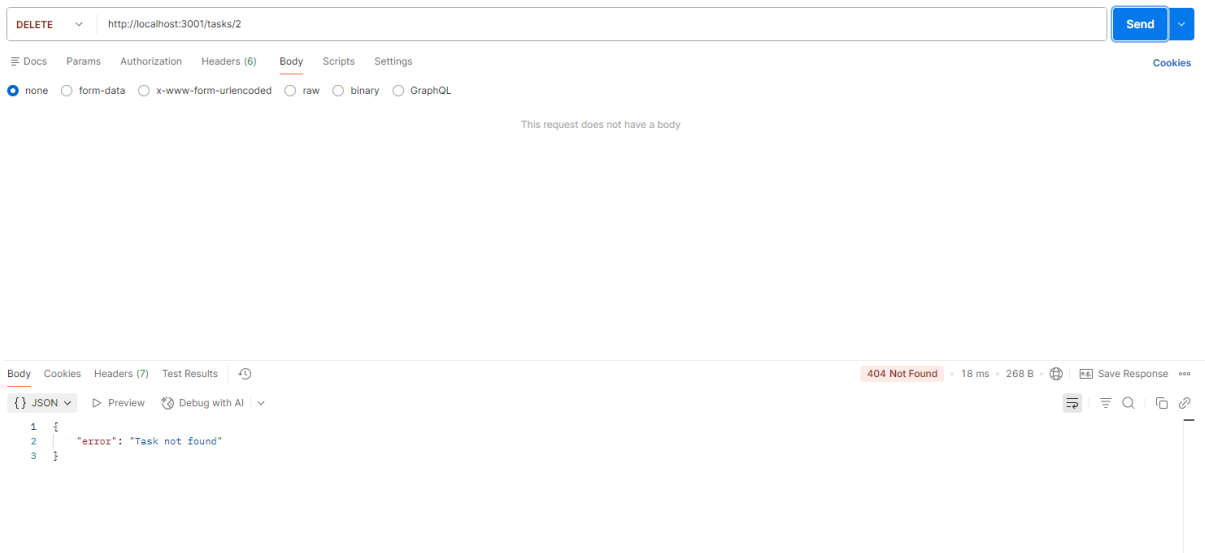
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Task not found' })
    }

    res.status(204).send()
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
})
```

borré la task 2, y da la respuesta 204:



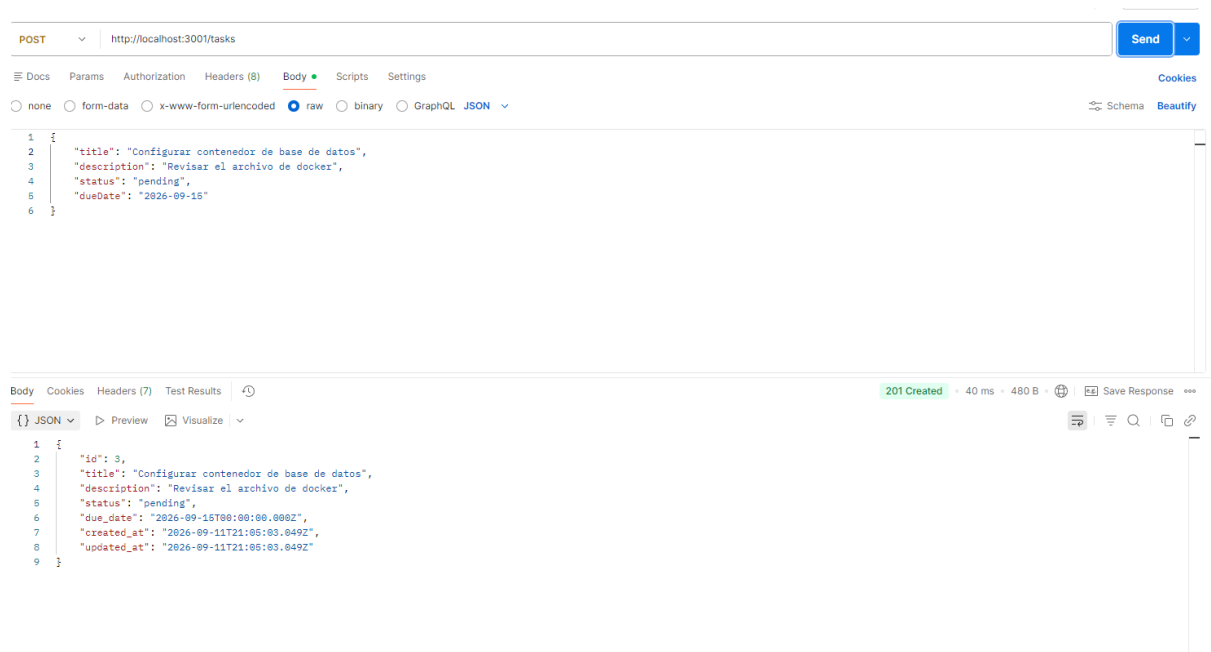
si la trato de volver a borrar, al no existir, te da 404:



10.4 Verificación de persistencia

1. Creá una tarea nueva con POST.
2. Reiniciá solo el contenedor de la API: `docker compose restart api`. Confirmá que la tarea sigue existiendo.
3. Ejecutá `docker compose down -v` y volvé a levantar el proyecto. Confirmá que esta vez sí se perdieron los datos, porque se eliminó el volumen.

se creó una task:



se reinició:

```
C:\Users\Gonza\Desktop\tps\Programación Avanzada\tps-programaci-n-avanzada\TP3>docker compose restart api
[+] restart 0/1
- Container todo_api Restarting
```

la task con id “3” sigue estando:

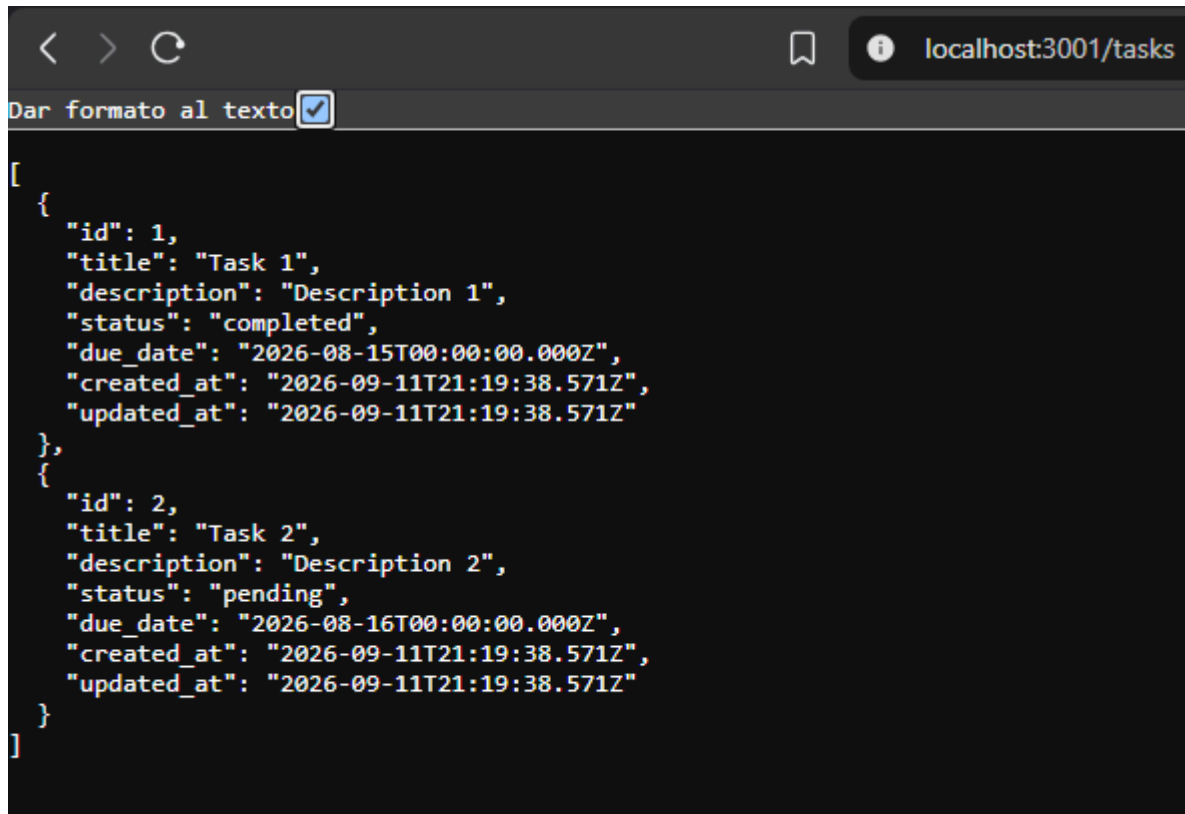
```
Dar formato al texto ☒

[
  {
    "id": 1,
    "title": "Task 1",
    "description": "Description 1",
    "status": "pending",
    "due_date": "2026-08-15T00:00:00.000Z",
    "created_at": "2026-09-10T20:04:25.660Z",
    "updated_at": "2026-09-11T20:55:16.263Z"
  },
  {
    "id": 3,
    "title": "Configurar contenedor de base de datos",
    "description": "Revisar el archivo de docker",
    "status": "pending",
    "due_date": "2026-09-15T00:00:00.000Z",
    "created_at": "2026-09-11T21:05:03.049Z",
    "updated_at": "2026-09-11T21:05:03.049Z"
  }
]
```

se paró completamente:

```
C:\Users\Gonza\Desktop\tps\Programación Avanzada\tps-programaci-n-avanzada\TP3>docker compose down -v
[+] down 4/4
✓Container todo_api      Removed
✓Container todo_db      Removed
✓Volume tp3_todo_pgdata Removed
✓Network tp3_default    Removed
```

se perdieron los datos:



The screenshot shows a web browser window with the address bar displaying 'localhost:3001/tasks'. Below the address bar is a toolbar with a refresh button and a bookmark icon. The main content area shows a REST client interface with a text input field containing 'Dar formato al texto' and a checked checkbox. Below this is a JSON array of two task objects. The first task has an id of 1, title 'Task 1', description 'Description 1', status 'completed', due_date '2026-08-15T00:00:00.000Z', created_at '2026-09-11T21:19:38.571Z', and updated_at '2026-09-11T21:19:38.571Z'. The second task has an id of 2, title 'Task 2', description 'Description 2', status 'pending', due_date '2026-08-16T00:00:00.000Z', created_at '2026-09-11T21:19:38.571Z', and updated_at '2026-09-11T21:19:38.571Z'.

```
[
  {
    "id": 1,
    "title": "Task 1",
    "description": "Description 1",
    "status": "completed",
    "due_date": "2026-08-15T00:00:00.000Z",
    "created_at": "2026-09-11T21:19:38.571Z",
    "updated_at": "2026-09-11T21:19:38.571Z"
  },
  {
    "id": 2,
    "title": "Task 2",
    "description": "Description 2",
    "status": "pending",
    "due_date": "2026-08-16T00:00:00.000Z",
    "created_at": "2026-09-11T21:19:38.571Z",
    "updated_at": "2026-09-11T21:19:38.571Z"
  }
]
```

Informe:

Se creó una task con un método POST, posteriormente se reinició sólo el contenedor de la API, usando docker compose restart api, al terminar de reiniciarse me fije y la task creada con POST seguía estando.

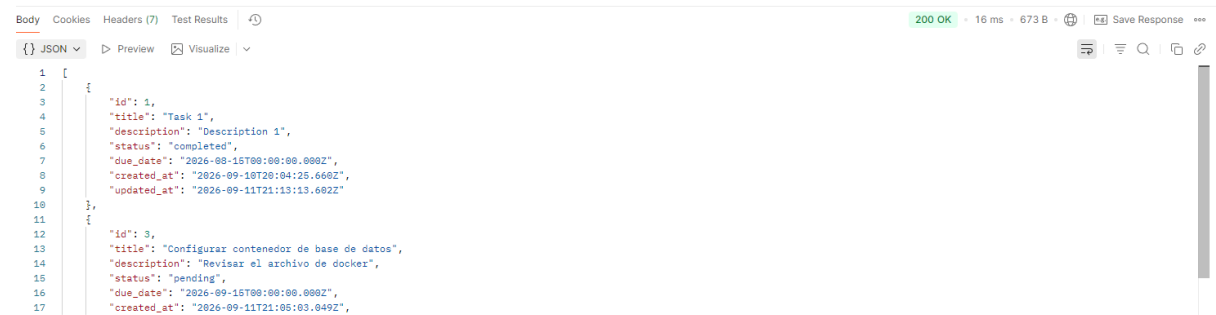
Posteriormente se probó parar el contenedor completamente con docker compose down -v y luego, al fijarme, me di cuenta que la task creada anteriormente había desaparecido.

Para evitar la pérdida de información, la configuración de la infraestructura utiliza Volúmenes de Docker (Docker Volumes). Un volumen es un mecanismo que permite alojar la información fuera del contenedor temporal, montando un directorio persistente directamente en la máquina host. De esta manera, el motor de Postgres guarda las tablas y registros en dicho volumen.

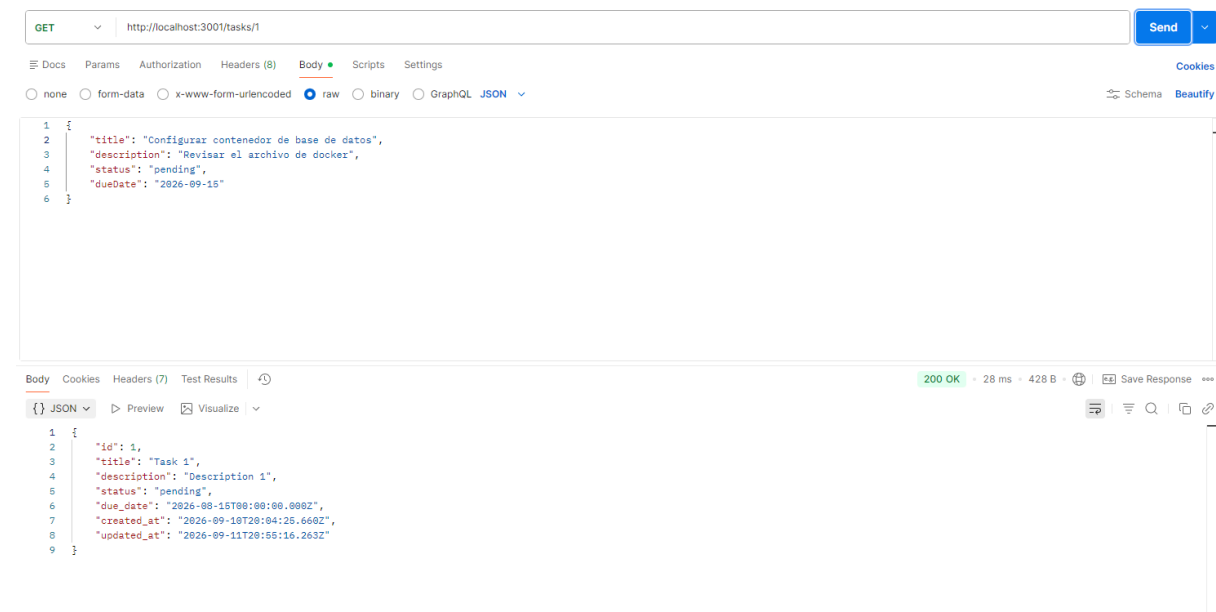
Los datos no se perdieron en la prueba porque están almacenados físicamente de forma independiente al ciclo de vida del contenedor. Mientras este volumen de Postgres no sea borrado explícitamente, la base de datos preservará su estado y la aplicación recuperará todas las tareas creadas en sesiones anteriores.

Prueba de los 5 endpoints:

GET(sin id):



GET(dando id):



si el id no existe:

GET http://localhost:3001/tasks/7 Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "title": "Configurar contenedor de base de datos",
3   "description": "Revisar el archivo de docker",
4   "status": "pending",
5   "dueDate": "2026-09-15"
6 }
```

Body Cookies Headers (7) Test Results 404 Not Found 18 ms 268 B Save Response

JSON Preview Debug with AI

```
1 {
2   "error": "Task not found"
3 }
```

POST:

POST http://localhost:3001/tasks Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "title": "Aprender sobre métodos HTTP",
3   "description": "Hacer pruebas con GET, POST, PUT y DELETE en Postman",
4   "status": "pending",
5   "dueDate": "2026-09-15"
6 }
```

Body Cookies Headers (7) Test Results 201 Created 25 ms 494 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 4,
3   "title": "Aprender sobre métodos HTTP",
4   "description": "Hacer pruebas con GET, POST, PUT y DELETE en Postman",
5   "status": "pending",
6   "due_date": "2026-09-15T08:00:00.000Z",
7   "created_at": "2026-09-11T21:11:17.057Z",
8   "updated_at": "2026-09-11T21:11:17.057Z"
9 }
```

si falta el título:

POST http://localhost:3001/tasks Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "title": "",
3   "description": "Hacer pruebas con GET, POST, PUT y DELETE en Postman",
4   "status": "pending",
5   "dueDate": "2026-09-15"
6 }
```

Body Cookies Headers (7) Test Results 400 Bad Request 7 ms 273 B Save Response

JSON Preview Debug with AI

```
1 {
2   "error": "Title is required"
3 }
```

PUT:

PUT

http://localhost:3001/tasks/1

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

1

{

2

"status": "completed"

3

}

Body

Cookies

Headers (7)

Test Results

200 OK

25 ms

430 B

Save Response

JSON

Preview

Visualize

1

{

2

"id": 1,

3

"title": "Task 1",

4

"description": "Description 1",

5

"status": "completed",

6

"due_date": "2026-09-15T00:00:00.000Z",

7

"created_at": "2026-09-10T20:04:25.660Z",

8

"updated_at": "2026-09-11T21:13:06.401Z"

9

}

si no existe una tarea con ese id:

PUT

http://localhost:3001/tasks/8

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

1

{

2

"status": "completed"

3

}

Body

Cookies

Headers (7)

Test Results

404 Not Found

17 ms

268 B

Save Response

JSON

Preview

Debug with AI

1

{

2

"error": "Task not found"

3

}

DELETE:

DELETE http://localhost:3001/tasks/4

Send

Docs Params Authorization Headers (6) **Body** Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (4) Test Results

204 No Content 33 ms 134 B Save Response

Raw Preview Visualize

1

si no existe el id:

DELETE http://localhost:3001/tasks/4

Send

Docs Params Authorization Headers (6) **Body** Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (7) Test Results

404 Not Found 17 ms 268 B Save Response

JSON Preview Debug with AI

```
1 {
2   "error": "Task not found"
3 }
```